CO2-AT2- Scenario Based Questions

Name: Ajay Kumar Reddy

Register.No:192425274

Batch: 2024

Course: CSA0712 - COMPUTER NETWORKS

Faculty: Dr. S. SALIN

Scenario 1:

A sender transmits **10 data frames** over a **high-noise wireless link**.

- **Frame 4 is lost** during transmission.
- **Frame 7 arrives with a bit error.**

1. How does a Stop-and-Wait ARQ protocol react when Frame 4 goes missing?

Answer:

In the **Stop-and-Wait ARQ** protocol, the sender transmits **one frame at a time** and waits for an **Acknowledgment (ACK)** before sending the next frame.

When **Frame 4 is lost**, the receiver does not receive it and therefore **does not send an ACK**. The sender waits until its **timer expires (timeout)**. After the timeout, the sender **retransmits Frame 4**. Once the receiver successfully receives Frame 4, it sends **ACK 4**, allowing the sender to continue transmitting the remaining frames.

Sequence

1. Sender sends Frame 4.
2. Frame 4 is lost.
3. Receiver sends no ACK.
4. Sender waits for timeout.
5. Timeout occurs.
6. Sender retransmits Frame 4.
7. Receiver sends ACK 4.
8. Transmission continues with Frame 5.

When Frame 4 goes missing, Stop-and-Wait ARQ waits for an acknowledgment. Since no ACK is received, the sender's timer expires, and Frame 4 is retransmitted. After receiving ACK 4, the sender continues sending the remaining frames.

2. What happens to Frames 5 and 6 if Stop-and-Wait is used?

Answer:

In **Stop-and-Wait ARQ**, only **one frame can be outstanding at any time**.

Since Frame 4 has not been acknowledged, the sender **cannot transmit Frame 5 or Frame 6**. These frames remain in the sender's buffer until Frame 4 is successfully retransmitted and acknowledged.

Frames 5 and 6 are not transmitted until Frame 4 is successfully retransmitted and ACK 4 is received. The sender waits because Stop-and-Wait allows only one unacknowledged frame at a time.

3. How does Go-Back-N ARQ handle the lost Frame 4 compared to Selective Repeat ARQ?

Go-Back-N ARQ

Answer

Go-Back-N ARQ allows the sender to transmit multiple frames without waiting for individual acknowledgments.

When **Frame 4 is lost**, the receiver successfully receives Frames 1–3 but expects Frame 4 next. Any subsequent frames (Frames 5–10) that arrive are considered **out of order** and are discarded. The receiver repeatedly acknowledges the last correctly received frame. After the sender's timeout, it retransmits **Frame 4 and all subsequent frames (Frames 5–10)**, including the corrupted Frame 7.

In Go-Back-N ARQ, the receiver discards Frames 5–10 after Frame 4 is lost. The sender retransmits Frame 4 and every frame that follows it (Frames 4–10).

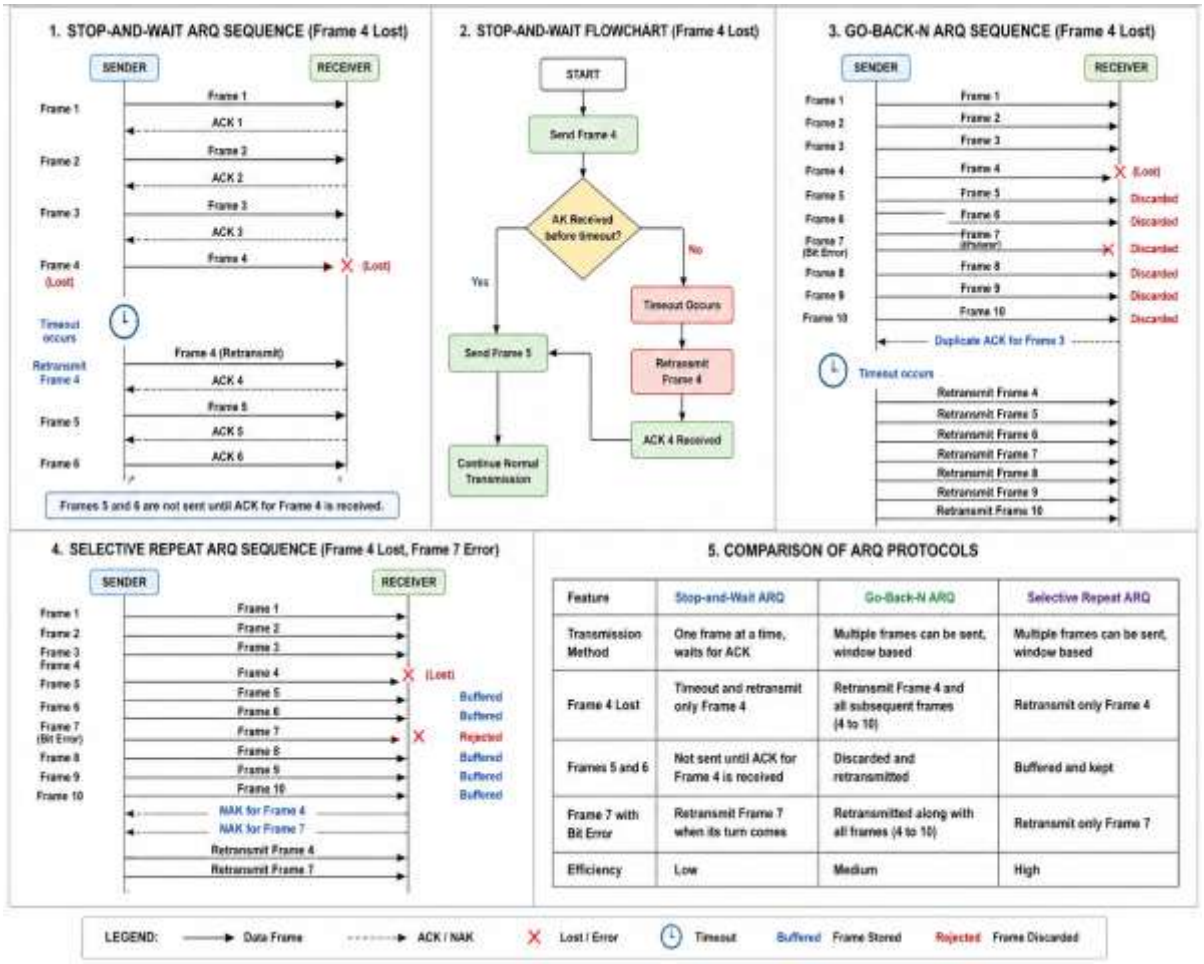
Selective Repeat ARQ

Answer

Selective Repeat ARQ also allows multiple frames to be transmitted, but the receiver can **accept and buffer out-of-order frames**.

When **Frame 4 is lost**, the receiver stores correctly received Frames 5, 6, 8, 9, and 10 in its buffer. However, **Frame 7 contains a bit error**, so it is rejected. The receiver requests retransmission only for the **missing Frame 4** and the **corrupted Frame 7**.

In Selective Repeat ARQ, only the lost Frame 4 and the corrupted Frame 7 are retransmitted. Correctly received frames (5, 6, 8, 9, and 10) are buffered and are not retransmitted, making the protocol more efficient than Go-Back-N.


1. STOP-AND-WAIT ARQ SEQUENCE (Frame 4 Lost)
SENDER
RECEIVER
Frame 1
ACK 1
Frame 2
ACK 2
Frame 3
ACK 3
Frame 4
(Lost)
Timeout occurs
Retransmit Frame 4
Frame 4 (Retransmit)
ACK 4
Frame 5
ACK 5
Frame 6
ACK 6
Frames 5 and 6 are not sent until ACK for Frame 4 is received.
2. STOP-AND-WAIT FLOWCHART (Frame 4 Lost)
START
Send Frame 4
ACK Received before timeout?
Yes
No
Timeout Occurs
Retransmit Frame 4
ACK 4 Received
Send Frame 5
Continue Normal Transmission
3. GO-BACK-N ARQ SEQUENCE (Frame 4 Lost)
SENDER
RECEIVER
Frame 1
Frame 2
Frame 3
Frame 4
(Lost)
Frame 5
Discarded
Frame 6
Discarded
Frame 7
(Bit Error)
Discarded
Frame 8
Discarded
Frame 9
Discarded
Frame 10
Discarded
Duplicate ACK for Frame 3
Timeout occurs
Retransmit Frame 4
Retransmit Frame 5
Retransmit Frame 6
Retransmit Frame 7
Retransmit Frame 8
Retransmit Frame 9
Retransmit Frame 10
4. SELECTIVE REPEAT ARQ SEQUENCE (Frame 4 Lost, Frame 7 Error)
SENDER
RECEIVER
Frame 1
Frame 2
Frame 3
Frame 4
(Lost)
Frame 5
Buffered
Frame 6
Buffered
Frame 7
(Bit Error)
Rejected
Frame 8
Buffered
Frame 9
Buffered
Frame 10
Buffered
NAK for Frame 4
NAK for Frame 7
Retransmit Frame 4
Retransmit Frame 7
5. COMPARISON OF ARQ PROTOCOLS
Feature
Stop-and-Wait ARQ
Go-Back-N ARQ
Selective Repeat ARQ
Transmission Method
One frame at a time, waits for ACK
Multiple frames can be sent, window based
Multiple frames can be sent, window based
Frame 4 Lost
Timeout and retransmit only Frame 4
Retransmit Frame 4 and all subsequent frames (4 to 10)
Retransmit only Frame 4
Frames 5 and 6
Not sent until ACK for Frame 4 is received
Discarded and retransmitted
Buffered and kept
Frame 7 with Bit Error
Retransmit Frame 7 when its turn comes
Retransmitted along with all frames (4 to 10)
Retransmit only Frame 7
Efficiency
Low
Medium
High
LEGEND:
Data Frame
ACK / NAK
Lost / Error
Timeout
Buffered Frame Stored
Rejected Frame Discarded

Scenario 2: MAC Sublayer and Collisions

Scenario:
Three office computers (**A, B, and C**) are connected to the same **Ethernet Hub**. Computers **A** and **B** begin transmitting data **at exactly the same time**.

1. What network event happens when both signals meet on the wire?

Answer

When Computers **A** and **B** transmit at the same time over the shared Ethernet hub, both electrical signals travel through the same communication medium. Since the hub is a **shared medium**, the two signals overlap.

This overlap creates a **collision**, causing both data frames to become corrupted. As a result, neither frame can be received correctly by the destination.

Sequence

1. Computer A senses the channel is idle.
2. Computer B also senses the channel is idle.
3. Both begin transmitting simultaneously.
4. Their signals meet on the Ethernet cable.
5. A **collision** occurs.
6. The transmitted frames become corrupted.

When both signals meet on the shared Ethernet cable, a collision occurs. The overlapping signals corrupt both frames, so neither transmission is successful.

2. How does CSMA/CD (Carrier Sense Multiple Access with Collision Detection) detect this event?

Answer

CSMA/CD stands for:

- **Carrier Sense** – Listen before transmitting.
- **Multiple Access** – Multiple devices share the same cable.
- **Collision Detection** – Detect if two devices transmit simultaneously.

After starting transmission, each computer continues monitoring the cable.

If the transmitted signal is different from the signal detected on the cable (because another device is transmitting simultaneously), the device recognizes that a collision has occurred.

Both computers immediately stop transmitting and send a **Jam Signal** so that every device connected to the hub knows a collision has occurred.

Sequence

1. A and B start transmitting.
2. Both monitor the cable.
3. Signal mismatch is detected.
4. Collision is detected.
5. Jam signal is transmitted.
6. Both computers stop transmitting.

CSMA/CD detects a collision by comparing the transmitted signal with the signal present on the cable. If they differ, the devices detect a collision, send a jam signal, and stop transmitting immediately.

3. Why do A and B wait for a random time period before they try to send data again?

Answer

If both computers retransmitted immediately after the collision, they would collide again.

To prevent repeated collisions, CSMA/CD uses the **Binary Exponential Backoff Algorithm**.

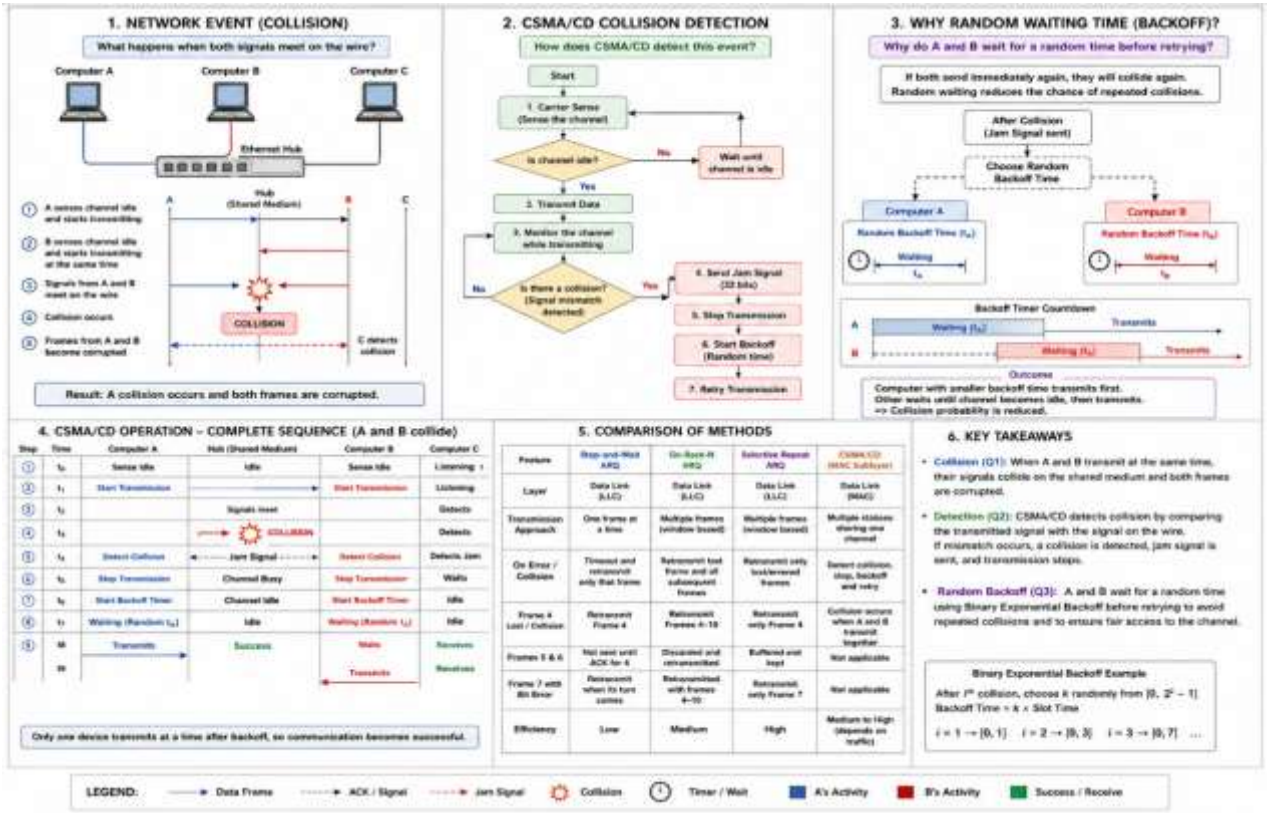
Each computer waits for a **random backoff time** before retransmitting.

Because the waiting time is random, there is a high probability that one computer transmits first while the other waits, allowing successful communication.

Sequence

1. Collision occurs.
2. Jam signal is sent.
3. Both computers stop transmitting.
4. Each computer selects a random backoff time.
5. The computer with the shorter waiting time transmits first.
6. The other computer transmits after the channel becomes free.

A and B wait for a random time period to reduce the chance of another collision. The random delay ensures that both devices do not retransmit simultaneously, improving network efficiency.


1. NETWORK EVENT (COLLISION)
What happens when both signals meet on the wire?
Computer A
Computer B
Computer C
Ethernet Hub
Hub (Shared Medium)
A
B
C
A senses channel idle and starts transmitting
B senses channel idle and starts transmitting at the same time
Signals from A and B meet on the wire
Collision occurs
Frames from A and B become corrupted
COLLISION
C detects collision
Result: A collision occurs and both frames are corrupted.
2. CSMA/CD COLLISION DETECTION
How does CSMA/CD detect this event?
Start
1. Carrier Sense (Sense the channel)
Is channel idle?
No
Wait until channel is idle
Yes
2. Transmit Data
3. Monitor the channel while transmitting
Is there a collision? (Signal mismatch detected)
No
Yes
4. Send Jam Signal (32 bits)
5. Stop Transmission
6. Start Backoff (Random time)
7. Retry Transmission
3. WHY RANDOM WAITING TIME (BACKOFF)?
Why do A and B wait for a random time before retrying?
If both send immediately again, they will collide again.
Random waiting reduces the chance of repeated collisions.
After Collision (Jam Signal sent)
Choose Random Backoff Time
Computer A
Computer B
Random Backoff Time (t_{BA})
Random Backoff Time (t_{BB})
Waiting
t_A
Waiting
t_B
Backoff Timer Countdown
A
Waiting (t_{BA})
Transmits
B
Waiting (t_{BB})
Transmits
Outcome
Computer with smaller backoff time transmits first.
Other waits until channel becomes idle, then transmits.
=> Collision probability is reduced.
4. CSMA/CD OPERATION – COMPLETE SEQUENCE (A and B collide)
Step
Time
Computer A
Hub (Shared Medium)
Computer B
Computer C
Sense Idle
Idle
Sense Idle
Listening
Start Transmission
Start Transmission
Listening
Signals meet
Detects
COLLISION
Detects
Detect Collision
Jam Signal
Detect Collision
Detects Jam
Stop Transmission
Channel Busy
Stop Transmission
Waits
Start Backoff Timer
Channel Idle
Start Backoff Timer
Idle
Waiting (Random t_{BA})
Idle
Waiting (Random t_{BB})
Idle
Transmits
Success
Waits
Receives
Transmits
Receives
Only one device transmits at a time after backoff, so communication becomes successful.
5. COMPARISON OF METHODS
Feature
Stop-and-Wait ARQ
Go-Back-N ARQ
Selective Repeat ARQ
CSMA/CD (IEEE 802.3 Ethernet)
Layer
Data Link (LLC)
Data Link (LLC)
Data Link (LLC)
Data Link (MAC)
Transmission Approach
One frame at a time
Multiple frames (window based)
Multiple frames (window based)
Multiple stations sharing one channel
On Error / Collision
Timeout and retransmit only that frame
Retransmit lost frame and all subsequent frames
Retransmit only lost/erroneous frames
Detect collision, stop, backoff and retry
Frame 4 Lost / Collision
Retransmit Frame 4
Retransmit Frames 4–10
Retransmit only Frame 4
Collision occurs when A and B transmit together
Frames 5 & 6
Not sent until ACK for 4
Discarded and retransmitted
Buffered and kept
Not applicable
Frame 7 with Bit Error
Retransmit when its turn comes
Retransmitted with frames 4–10
Retransmit only Frame 7
Not applicable
Efficiency
Low
Medium
High
Medium to High (depends on traffic)
6. KEY TAKEAWAYS
Collision (Q1): When A and B transmit at the same time, their signals collide on the shared medium and both frames are corrupted.
Detection (Q2): CSMA/CD detects collision by comparing the transmitted signal with the signal on the wire. If mismatch occurs, a collision is detected, jam signal is sent, and transmission stops.
Random Backoff (Q3): A and B wait for a random time using Binary Exponential Backoff before retrying to avoid repeated collisions and to ensure fair access to the channel.
Binary Exponential Backoff Example
After i^{th} collision, choose k randomly from $[0, 2^i - 1]$
Backoff Time = $k \times$ Slot Time
$i = 1 \rightarrow [0, 1]$ $i = 2 \rightarrow [0, 3]$ $i = 3 \rightarrow [0, 7]$ ...
LEGEND:
Data Frame
ACK / Signal
Jam Signal
Collision
Timer / Wait
A's Activity
B's Activity
Success / Receive

Scenario 3: Flow Control and Buffer Overflow

Scenario

A **fast server** sends data frames to an **older, slow printer**. The printer has a **very small memory buffer** for incoming data.

1. What problem happens if the server sends frames faster than the printer can clear its buffer?

Answer

The printer can process data only at a limited speed. If the server continues sending frames faster than the printer can process them, the printer's **buffer (temporary memory)** gradually becomes full.

Once the buffer is full, it cannot accept any more incoming frames. The additional frames are **dropped (lost)**, causing **buffer overflow**. Lost frames must be retransmitted, resulting in increased network traffic, delay, and reduced efficiency.

Sequence

1. Server sends frames continuously.
2. Printer stores frames in its buffer.
3. Printer processes frames slowly.
4. Buffer becomes full.
5. New incoming frames are discarded.
6. Buffer overflow occurs.
7. Lost frames must be retransmitted.

If the server sends data faster than the printer can process it, the printer's buffer becomes full, causing buffer overflow. Extra frames are lost and must be retransmitted, leading to delays and reduced network performance.

2. How does a sliding window flow control protocol stop the printer's buffer from overflowing?

Answer

Sliding Window Flow Control ensures that the sender transmits only a limited number of frames before waiting for acknowledgements (ACKs).

The printer informs the server how much free buffer space is available by advertising its **window size**. The server sends only the number of frames allowed by the current window.

As the printer processes data, it frees buffer space and sends a new ACK with an updated window size. The server then adjusts its transmission rate accordingly.

Sequence

1. Printer advertises available window size.
2. Server sends only the allowed number of frames.

3. Printer stores frames in its buffer.
4. Printer processes the buffered frames.
5. Printer sends ACK with updated window size.
6. Server sends additional frames based on the new window size.
7. Buffer never overflows.

Sliding Window Flow Control prevents buffer overflow by limiting the number of unacknowledged frames the server can send. The server transmits only the amount of data that the printer's available buffer can handle.

3. What role does the window size field in the acknowledgement frame play here?

Answer

The **Window Size field** in the ACK frame tells the sender how much free buffer space is currently available in the receiver.

This value acts as a feedback mechanism.

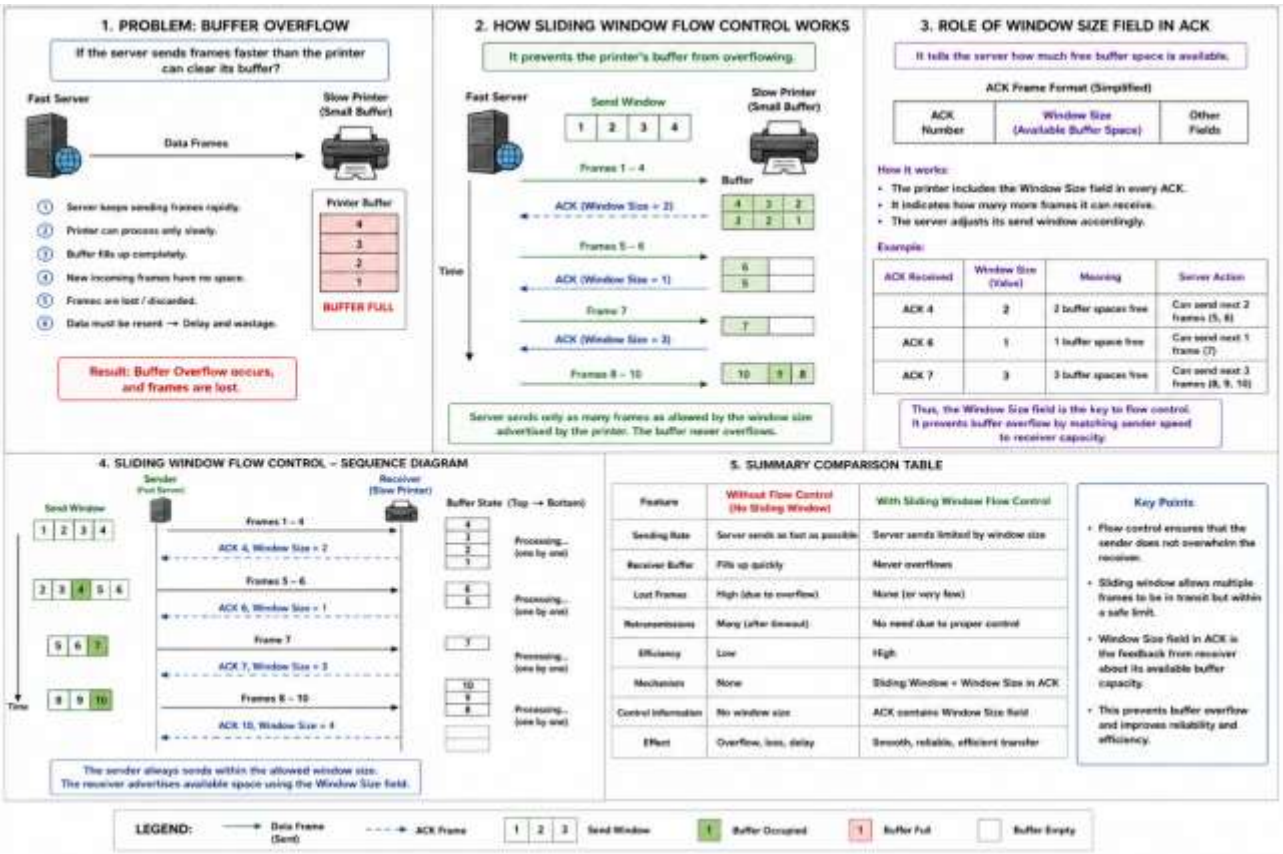
- If the window size is **large**, the server can send more frames.
- If the window size is **small**, the server slows down and sends fewer frames.
- If the window size becomes **zero**, the server temporarily stops sending until the printer has available buffer space again.

Thus, the sender always matches its transmission speed with the receiver's processing capacity.

Example

ACK Received	**Window Size**	**Meaning**	**Server Action**
ACK 4	2	Two buffer spaces available	Send Frames 5 and 6
ACK 6	1	One buffer space available	Send Frame 7 only
ACK 7	3	Three spaces available	Send Frames 8, 9 and 10
ACK 10	0	Buffer full	Stop sending until new ACK arrives

The Window Size field tells the sender how many additional frames the receiver can accept. The sender adjusts its transmission rate based on this value, preventing the printer's buffer from overflowing.


1. PROBLEM: BUFFER OVERFLOW
If the server sends frames faster than the printer can clear its buffer?
Fast Server
Slow Printer (Small Buffer)
Data Frames
Server keeps sending frames rapidly.
Printer can process only slowly.
Buffer fills up completely.
New incoming frames have no space.
Frames are lost / discarded.
Data must be resent → Delay and wastage.
Printer Buffer
4
3
2
1
BUFFER FULL
Result: Buffer Overflow occurs, and frames are lost.
2. HOW SLIDING WINDOW FLOW CONTROL WORKS
It prevents the printer's buffer from overflowing.
Fast Server
Send Window
1 2 3 4
Slow Printer (Small Buffer)
Buffer
Frames 1 – 4
ACK (Window Size = 2)
4 3 2
3 2 1
Frames 5 – 6
ACK (Window Size = 1)
6
5
Frame 7
ACK (Window Size = 3)
7
Frames 8 – 10
10 9 8
Time
Server sends only as many frames as allowed by the window size advertised by the printer. The buffer never overflows.
3. ROLE OF WINDOW SIZE FIELD IN ACK
It tells the server how much free buffer space is available.
ACK Frame Format (Simplified)
ACK Number
Window Size (Available Buffer Space)
Other Fields
How it works:
The printer includes the Window Size field in every ACK.
It indicates how many more frames it can receive.
The server adjusts its send window accordingly.
Example:
ACK Received
Window Size (Value)
Meaning
Server Action
ACK 4
2
2 buffer spaces free
Can send next 2 frames (5, 6)
ACK 6
1
1 buffer space free
Can send next 1 frame (7)
ACK 7
3
3 buffer spaces free
Can send next 3 frames (8, 9, 10)
Thus, the Window Size field is the key to flow control. It prevents buffer overflow by matching sender speed to receiver capacity.
4. SLIDING WINDOW FLOW CONTROL – SEQUENCE DIAGRAM
Sender
Receiver (Slow Printer)
Send Window
1 2 3 4
Frames 1 – 4
ACK 4, Window Size = 2
2 3 4 5 6
Frames 5 – 6
ACK 6, Window Size = 1
5 6 7
Frame 7
ACK 7, Window Size = 3
8 9 10
Frames 8 – 10
ACK 10, Window Size = 4
Time
Buffer State (Top → Bottom)
4 3 2 1
Processing... (one by one)
6 5
Processing... (one by one)
7
Processing... (one by one)
10 9 8
Processing... (one by one)
The sender always sends within the allowed window size. The receiver advertises available space using the Window Size field.
5. SUMMARY COMPARISON TABLE
Feature
Without Flow Control (No Sliding Window)
With Sliding Window Flow Control
Sending Rate
Server sends as fast as possible
Server sends limited by window size
Receiver Buffer
Fills up quickly
Never overflows
Lost Frames
High (due to overflow)
None (or very few)
Retransmissions
Many (after timeout)
No need due to proper control
Efficiency
Low
High
Mechanism
None
Sliding Window + Window Size in ACK
Control Information
No window size
ACK contains Window Size field
Effect
Overflow, loss, delay
Smooth, reliable, efficient transfer
Key Points
Flow control ensures that the sender does not overwhelm the receiver.
Sliding window allows multiple frames to be in transit but within a safe limit.
Window Size field in ACK is the feedback from receiver about its available buffer capacity.
This prevents buffer overflow and improves reliability and efficiency.
LEGEND:
Data Frame (Sent)
ACK Frame
1 2 3
Send Window
1
Buffer Occupied
1
Buffer Full
Buffer Empty

Scenario 4: Error Detection with CRC

Scenario:
A device uses the **Generator Polynomial**:

Binary form:

1011

Data bit string:

110101

1. How many redundant bits (CRC bits) are appended to the data frame?

Answer

The number of CRC bits appended is equal to the **degree of the generator polynomial**.

Generator Polynomial:

Highest power of **x = 3**

Therefore,

Number of CRC bits = 3

Calculation

Generator = 1011

Degree = 3

CRC bits appended = 3

3 redundant (CRC) bits are appended to the original data frame.

2. Walk through the modulo-2 division steps to find the final transmitted bit sequence.

Step 1: Original Data

110101

Append **3 zeros** (because CRC = 3 bits)

110101000

Step 2: Generator

1011

Step 3: Modulo-2 Division

111

_____

1011) 110101000

Perform XOR division (instead of normal subtraction).

1101 XOR 1011 = 0110

Bring down next bit

1110 XOR 1011 = 0101

Bring down next bit

1010 XOR 1011 = 0001

Bring down next bit

Continue until all bits are processed

Final Remainder = 111

Therefore,

CRC = 111

Step 4: Final Transmitted Frame

Original Data

110101

CRC

111

Final transmitted sequence

110101111

CRC Remainder = 111

Final transmitted bit sequence = 110101111

3. If a burst error flips the first two bits during transmission, how does the receiver find out that the frame is corrupted?

Answer

Suppose the transmitted frame is

110101111

A burst error flips the first two bits.

Received frame becomes

010101111

The receiver divides the received frame by the same generator polynomial (**1011**).

010101111 ÷ 1011

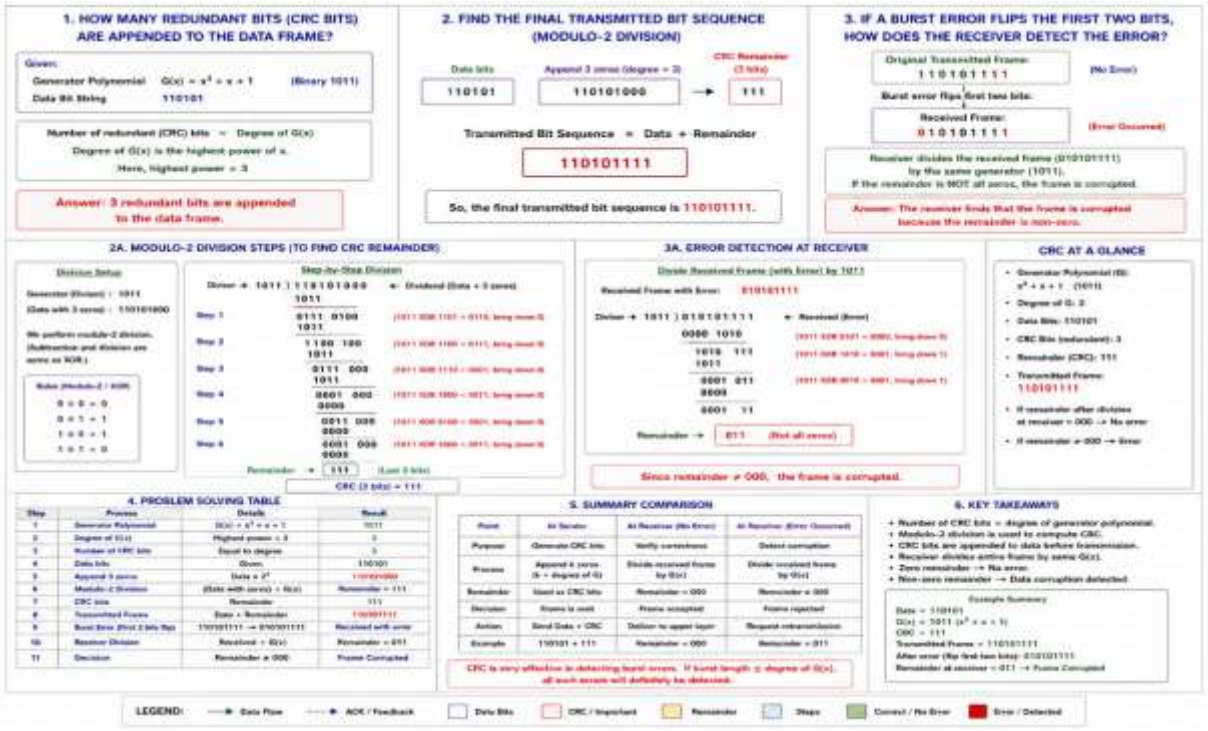
The receiver obtains the remainder

011

Since the remainder is **not equal to 000**, the CRC check fails.

Therefore, the receiver concludes that the frame has been corrupted during transmission.

The receiver divides the received frame by the generator polynomial (1011). Since the remainder is non-zero (011), the receiver detects that the frame is corrupted and rejects it (or requests retransmission, depending on the protocol).



Scenario : 4

1. How many Redudant Bits are appered to the data.

Given

Generator Polynomial G(x) = $x^3 + x + 1$ (Binary = 1011)

Data Bit string 110101

No. of Redundant bits = Degree of G(x)

Degree of G(x) = highest power of x

Highest power of x = 3

So 3 redudant bits are appended to the data frame.

2. Find Final Transmitted Bit Sequence

```
1011) 1 1 0 1 0 1 0 0 0
      1 0 1 1
      -------
      0 1 1 1 0 1 0 0
        1 0 1 1
        -------
        0 1 1 1 0 0 0
          1 0 1 1
          -------
          0 0 0 1 0 0 0
            0 0 0 0
            -------
            0 0 1 1 0 0 0
              0 0 0 0
              -------
              0 1 1 0 0 0
                0 0 0 0
                -------
                1 1 0 0 0
                  1 0 1 1
                  -------
                  1 1 1
```

Remainder = 111

3. Error detection

```
1011 ) 0 1 0 1 0 1 1 1 1
       0 0 0 0
       ---------
         1 0 1 0 1 1 1 1
         1 0 1 1
         ---------
           0 0 0 1 1 1 1
             0 0 0 0
             ---------
               0 0 1 1
```

Remainder = 011

Remainder ≠ 000 → the frame is corrupted

Scenario 5: Framing and Byte Stuffing

Scenario

A communication link uses the special **flag byte 0x7E** to mark the **start** and **end** of every frame. However, the payload data itself contains the byte **0x7E** inside a text message.

1. Why does the receiver think the frame ended early when it reads the data payload?

Answer

In byte-oriented protocols, **0x7E** is reserved as a **Flag byte** to indicate the beginning and end of a frame.

If the same byte (**0x7E**) appears inside the payload data, the receiver cannot distinguish whether it is a data byte or the end-of-frame flag. As a result, the receiver mistakenly interprets the data byte as the **end of the frame**, causing the frame to terminate prematurely. The remaining data is treated as another frame or discarded.

Sequence

1. Sender starts the frame with **0x7E**.
2. Payload contains **0x7E**.
3. Receiver reads the payload.
4. Receiver mistakes the payload **0x7E** for the end flag.
5. Frame ends early.
6. Remaining data becomes invalid or is discarded.

Example

Without Byte Stuffing

Frame Sent

|7E| A1 | B2 | 7E | C3 | D4 | 7E|

Receiver Reads

Start → A1 → B2 → END

Remaining Data

C3 D4 7E

(Frame Lost)

The receiver thinks the frame has ended because the byte 0x7E inside the payload is identical to the flag byte used to mark the end of a frame. This causes premature frame termination and incorrect interpretation of the remaining data.

2. How does byte stuffing (escape character insertion) solve this issue?

Answer

Byte Stuffing solves this problem by inserting a special **Escape byte (0x7D)** before every occurrence of **0x7E** or **0x7D** in the payload.

The sender also modifies the original byte using **XOR with 0x20**.

When the receiver encounters **0x7D**, it recognizes that the next byte is **data**, not a flag. It removes the escape byte and reverses the XOR operation to recover the original byte.

Thus, the receiver never mistakes data for a frame delimiter.

Sequence

1. Sender scans the payload.
2. Finds **0x7E** or **0x7D**.
3. Inserts Escape byte **0x7D**.
4. Performs **XOR with 0x20**.
5. Receiver detects **0x7D**.
6. Receiver removes **0x7D**.
7. Receiver reverses XOR.
8. Original data is restored.

Byte stuffing prevents framing errors by inserting an escape byte (0x7D) before any occurrence of 0x7E or 0x7D in the payload. This allows the receiver to distinguish data bytes from frame delimiter bytes.

3. Show how the sender transforms the data byte 0x7E and the escape byte itself before sending.

Transformation Rules

Original Byte	Meaning	Transmitted Bytes
0x7E	Flag Byte	**0x7D 0x5E**
0x7D	Escape Byte	**0x7D 0x5D**

How the Transformation Works

Case 1: Data Byte = 0x7E

Original Byte

0x7E

↓

Insert Escape Byte

0x7D

↓

0x7E XOR 0x20 = 0x5E

↓

Transmitted

0x7D 0x5E

Case 2: Data Byte = 0x7D

Original Byte

0x7D

↓

Insert Escape Byte

0x7D

↓

0x7D XOR 0x20 = 0x5D

↓

Transmitted

0x7D 0x5D

Example

Original Payload

A1 7E B2 7D C3

After Byte Stuffing

A1 7D 5E B2 7D 5D C3

Complete Frame

7E | A1 7D 5E B2 7D 5D C3 | 7E

Receiver removes the escape bytes and reverses the XOR operation to recover:

A1 7E B2 7D C

The sender transforms special bytes as follows:

0x7E → 0x7D 0x5E

0x7D → 0x7D 0x5D

This ensures that special flag and escape bytes appearing in the payload are transmitted safely without being mistaken for frame delimiters.

